

Next-Generation Retail Intelligence through Vision-Driven Product Recognition and Transaction Automation

Abstract. Retail checkout continues to lean significantly on barcode scanning, where each item needs to be managed individually and readable code is required; this becomes inefficient when a variety of products are presented together or upon label degradation, hidden, or complex to read. Although recent vision-based retail studies report strong product-detection performance, many stop at recognition and give less attention to the complete path from detection to billing, including background false positives, database retrieval, and operator correction. This work presents a vision-driven grocery checkout framework that combines image-based multi-product detection with GTIN-based identification, database retrieval, and a human-in-the-loop POS interface. A custom ten-product dataset was prepared under varied lighting, backgrounds, orientations, and viewing distances, while 154 background-only scenes and mixed RGB/grayscale inputs were included to examine robustness. YOLOv8m, YOLOv11m, YOLOv12m, and RT-DETR were compared using detection accuracy, localization quality, inference latency, model size, and background-error behaviour. All four detectors achieved 99.4–99.5% mAP@0.5. YOLOv8m reached 93.5% mAP@0.5:0.95 with only two background false positives, whereas YOLOv11m was the fastest at 544.8 ms per image and the smallest at 38.6 MB. The findings show that checkout-oriented model selection requires a balance between localization accuracy, computational efficiency, and billing-sensitive false detections, supporting the proposed framework as a practical basis for snapshot-based assisted grocery checkout.

Keywords: Automated grocery checkout, multi-product detection, computer vision, YOLO, RT-DETR, GTIN-based identification, retail automation.

1 Introduction

Retail checkout is a time-sensitive task, so both the speed and the accuracy of product identification matter. Conventional barcode systems still depend on manual handling, visible code, correct placement, and one-by-one scanning. A degraded or concealed barcode, or even a change in packaging, can delay the transaction. Computer-vision-based detection offers another route because items can be identified immediately from camera images. Swift and more capable object identification has become possible with

recent innovations in deep learning. Within this domain, YOLO models have gained widespread adoption because of their fast reasoning speed, efficient framework, and suitability for real-world deployment. YOLOv8, in particular, has demonstrated a favorable trade-off between detection accuracy and computational cost [1,5]. Building on this foundation, YOLO-based methods have been applied to grocery recognition tasks and intelligent retail shelf management systems [2]. With each successive release, the YOLO family has continued to advance in accuracy, inference speed, and deployment flexibility [3], and among these, YOLOv11 has reported promising results specifically for grocery-product recognition [2]. In parallel, transformer-based detectors, such as RT-DETR have attracted growing interest, largely due to their capacity to model long-range visual dependencies and to reliably locate objects even in visually cluttered or complex scenes [4]. More recent studies comparing YOLOv11 and RT-DETR have highlighted the competitive potential of both architectures for grocery detection and broader retail applications [6].

These advances do not go far enough: the bulk of the studies conducted have focused on detection performance alone, and therefore only a small fraction of the requirements of a full checkout workflow. Of course, implementing such a system also requires attention to the quality of the data, the pre-processing of the data, the quality of the annotations, the latency of the inference, the size of the model, and the integration with the point-of-sale system. This is further complicated in retail environments because lighting conditions vary, products are oriented differently, are at different distances from the camera, have cluttered backgrounds, and there is visual similarity among packages of different classes

This work tackles that practical scenario with a vision-based grocery checkout system for ten retail product classes. The image acquisition, pre-processing, labeling, split of data into training, validation, and test sets, gray-scale augmentation, training, evaluating, and integrating the model into a point-of-sale system are all evaluated. To reduce false detections, background-only images are deliberately retained as negative examples. The performance of the model is evaluated according to mean average precision, precision, recall, inference latency, and model size, and the most efficient detector is then mapped to a Python-based prototype of a POS, taking the work beyond mere recognition to an end-to-end checkout system.

The dataset was collected under diverse backgrounds and lighting conditions. Product samples range from 274 to 286 per class, giving a closely balanced allocation, and 154 background-only samples are included as negative scenes. The main contributions of the work are:

- (i) An end-to-end, camera-based product recognition pipeline from data collection and pre-processing to annotation, model training, evaluation, selection, and application-level deployment.
- (ii) A consistent experimental setup for the evaluation of three models under identical, checkout-oriented conditions, where the three models are compared for the detection of grocery products—YOLOv8, YOLOv11, and RT-DETR.
- (iii) An evaluation that considers both detection-level metrics (mean average precision, precision and recall) as well as deployment-level metrics (inference latency, model size and memory requirements).
- (iv) Adding background-only negative samples, grayscale augmentation, and Python integration with POS, which are designed to enhance operational reliability, reduce the number of false positives due to background, and minimise the risk of billing in error.

The rest of the paper is organized as follows. **Section 2** discusses related work. **Section 3** describes the materials and methods, and **Section 4** presents the experimental results. The main findings are summarized in **Section 5**.

2 Literature Review

With the growing need for fast visual recognition, object detection is becoming an essential part of many real-time systems. During the practical operation of the device, reliable recognition can be maintained by using YOLOv8 in an automatic health monitoring system to read the device’s parameters pertaining to the device, as shown by Mahmood et al. [2]. YOLOv8 has also been adapted to other applications, such as the detection of objects in drone-based systems, as demonstrated by Maarooof and Bouhlel in [8]. Both studies were not set in a retail checkout environment, however, and neither considered issues like similar packages, overlapping products, reflective surfaces, or the type of false detections that can directly lead to billing errors.

More recent studies have focused on retail-related research, which has brought it closer to dealing with these practical issues. Thirumurthy et al. [3] implemented an intelligent shelf management system using transfer learning and the YOLOv8n lightweight detector, demonstrating the capability of a lightweight detector to support automated monitoring in retail for good merchandise. As the adoption of both convolutional and transformer-based detectors for product-recognition tasks continues to grow, Sathya et al. [7] compared YOLOv11 with RF-DETR for the task of grocery detection. However, shelf monitoring and grocery detection do not form a complete checkout process; most of these systems are not part of point-of-sale operations, quantity control, pricing information, or product identifiers.

The detection accuracy, inference speed, and deployment flexibility of the YOLO family have also been continuously improved. Hassani et al. [4] made a comparative study of YOLOv1 to YOLOv11, detailing the evolution of the architectures over generations of the model. Their results highlight the need to consider the computational costs and deployability of the model in addition to its accuracy for model selection, not only based on the accuracy of the model. The development of transformer-based detection has taken a very similar path, with a key focus on small object recognition and complex scenes. The RT-DETR architecture was modified by Madan and Reich [10] to better detect small objects, with an improvement in localization performance in visually challenging scenes. However, the trade-off for this gain is a higher computational demand and memory usage, which can be significant for a lightweight real-time RT-DETR deployment in resource-constrained systems.

The work presented here is a continuation of these studies, where the authors compare YOLOv8, YOLOv11, and RT-DETR in the end-to-end grocery checkout system, instead of the detection of products as a standalone task. Balanced product classes, background-only negative samples, grayscale augmentation, latency and model-size analysis, and Python-based POS integration are brought together, as well as the practical implications that false positives can have for billing accuracy, with a focus on detection reliability.

3 Materials and Methods

Fig. 1 summarizes the workflow used to build and evaluate the grocery checkout system. Work began by collecting images for ten grocery-product classes under different lighting conditions, viewing angles, distances, and backgrounds. Background-only images were added as negative scenes so the detectors also learned what a checkout image looks like when no target product is present. After collection, the images were preprocessed and annotated, then separated into training, validation, and test subsets before grayscale augmentation was applied. YOLOv8, YOLOv11, and RT-DETR were trained within the same experimental structure. Their outputs were compared in terms of detection

accuracy, false-positive behaviour, inference latency, model size, and computational requirements. The detector that offered the most useful overall trade-off was then connected to a Python-based POS application for product identification and billing assistance.

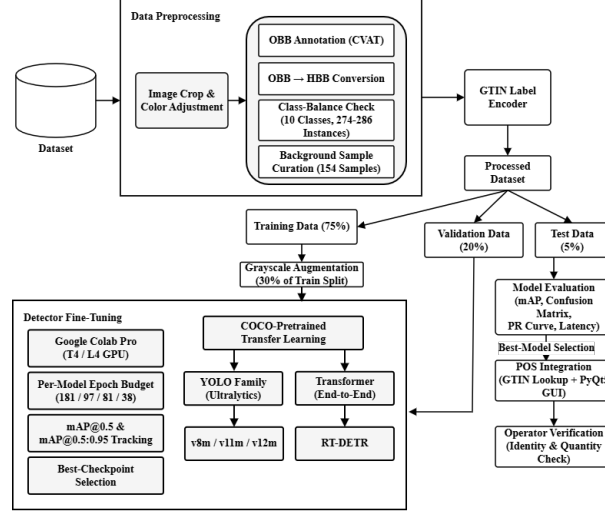


Fig. 1. Methodological Framework of the Proposed AI-Powered Grocery Checkout System

3.1 Dataset Description

The dataset was created specifically for grocery-product recognition in a retail-checkout setting. It contains ten product categories with closely matched numbers of annotated instances. Each class has between 274 and 286 product instances, giving 2,804 annotated product instances in total. A further 154 background-only samples represent scenes in which none of the target products is present.

Images were captured under different environmental conditions rather than using one fixed checkout arrangement. **Table 1** lists the main dataset characteristics. During collection, lighting intensity, product orientation, camera distance, background colour, package reflection, and product placement were varied. These changes were included because real checkout scenes may contain rotated or partly occluded products, closely placed items, reflections, and visually similar surrounding objects.

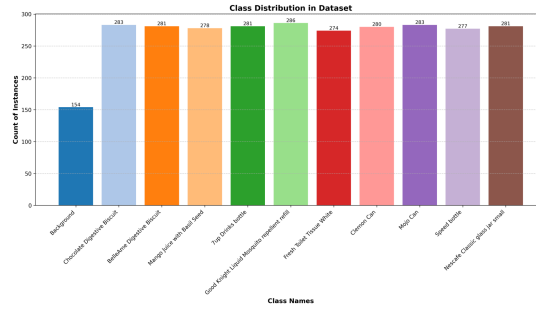
Fig. 2 shows how closely the ten target classes are distributed, with only a small difference between the lowest and highest class counts (274–286 instances). This reduces the chance that one product category dominates training simply because it appears more often. The 154 background-only samples provide negative checkout scenes and are important for checking spurious detections that could otherwise result in an incorrect item being added to a bill.

3.2 Image Collection and Preprocessing

Product images were taken with a camera positioned to resemble the view at a checkout counter. Products were photographed on their own and, where appropriate, together

Table 1. DATASET CHARACTERISTICS

Characteristic	Description
Dataset type	Custom grocery-product image dataset
Task	Multi-class object detection
Number of product classes	10
Product instances	2,804
Background-only samples	154
Product instances per class	274–286
Annotation type	Bounding boxes with product-class labels
Input conditions	Varied lighting, orientations, distances, and backgrounds
Models evaluated	YOLOv8, YOLOv11, and RT-DETR
Deployment platform	Python-based POS application

**Fig. 2.** Distribution of Annotated Instances Across the Ten Grocery-Product Classes and Background Samples

with other products. The camera distance and viewing angle were changed across captures so that the dataset did not represent only one fixed presentation.

Before annotation and training, the images went through basic preprocessing. Rotation, brightness adjustment, contrast correction, and image-format standardisation were used to reduce inconsistencies introduced during image capture while keeping the visual information needed for recognition.

These operations were kept conservative. Strong editing could change package colours or remove details that distinguish one product from another, so the original images were retained and the modified versions were added as extra training examples rather than replacing the originals.

3.3 Image Annotation and Dataset Partitioning

Each target object was assigned a bounding box together with its product class or identifier. Some annotations were initially prepared as oriented bounding boxes; these were converted to horizontal boxes so that the same labels could be used with the selected detection frameworks. The annotations were then checked manually for wrong class labels, incomplete boxes, and boxes that extended beyond the product boundary.

The prepared data were divided into training, validation, and test subsets in a 75:20:5 ratio. Training data were used to update the model parameters, validation data were used to follow training behaviour and support model selection, and the test subset was kept separate for the final performance evaluation.

Partitioning was done before augmentation. Images derived from the same original photograph were kept in the same subset, preventing an original image and its modified version from appearing on opposite sides of the train/test boundary. This step reduces the risk of data leakage and an overly optimistic estimate of detection performance. [Table 2](#) gives the split and the role of each subset.

Table 2. DATASET PARTITIONING AND PURPOSE

Subset	Proportion	Purpose
Training	75%	Model learning and parameter optimisation
Validation	20%	Training monitoring and model selection
Test	5%	Final independent performance evaluation

3.4 Grayscale Augmentation

Grayscale conversion was added as a controlled augmentation step so that the detectors would not depend entirely on package colour. Thirty per cent of the training images and thirty per cent of the validation images were converted to grayscale separately; the remaining images kept their original colour information.

Using both representations exposed the models to colour cues as well as structural information. Grayscale samples can be useful when colour is weakened by lighting, reflections, or camera settings, but colour is also important for separating some grocery packages. For that reason, grayscale conversion was applied to only part of the data rather than to the complete dataset.

All augmentation took place after the dataset had already been partitioned. As a result, an augmented copy originating from a training image could not move into the validation or test subset.

3.5 Model Training and Comparative Evaluation

YOLOv8, YOLOv11, and RT-DETR were trained independently on the same product classes and dataset partitions. YOLOv8 and YOLOv11 are single-stage detectors intended for efficient localization and classification, whereas RT-DETR uses a transformer-based detection design that can model wider visual relationships between objects.

The comparison used the same training, validation, and test partitions for every model. Experimental factors such as image resolution, number of epochs, batch size, hardware, and evaluation thresholds were intended to remain common wherever the model implementations allowed it. When an architecture required a model-specific setting, that difference was to be recorded rather than hidden. **Table 3** summarizes the training configuration used for the comparison.

Table 3. MODEL TRAINING CONFIGURATION

Parameter	YOLOv8	YOLOv11	RT-DETR
Input image size	Report actual value	Report actual value	Report actual value
Batch size	Report actual value	Report actual value	Report actual value
Number of epochs	Report actual value	Report actual value	Report actual value
Initial learning rate	Report actual value	Report actual value	Report actual value
Optimiser	Report actual value	Report actual value	Report actual value
Confidence threshold	Report actual value	Report actual value	Report actual value
IoU threshold	Report actual value	Report actual value	Report actual value
Training hardware	Report actual value	Same hardware	Same hardware

3.6 Performance Metrics

Evaluation covered both detection quality and deployment-related behaviour. Precision, recall, average precision, mean average precision, inference latency, model size, and memory requirements were considered. This combination is important in checkout use because a detector must not only find products correctly; it should also avoid false detections that can place an item on the bill when that item is not present.

Intersection over Union Intersection over Union (IoU) describes how much a predicted box overlaps the corresponding ground-truth box:

$$\text{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \quad (1)$$

where B_p represents the predicted bounding box and B_g represents the ground-truth bounding box.

Precision Precision indicates what proportion of the predicted product detections are correct:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (2)$$

where TP denotes true-positive detections and FP denotes false-positive detections. Precision is especially important at checkout because a false-positive prediction can add a product that is not actually present.

Recall Recall measures how many of the products that are actually present are detected successfully:

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (3)$$

where FN denotes false-negative detections. A low recall value means that some products can be missed during checkout.

Average Precision For each product class, Average Precision (AP) is obtained from the area under its precision–recall curve:

$$AP_c = \int_0^1 P_c(R) dR. \quad (4)$$

where $P_c(R)$ is the precision of class c at a given recall level.

Mean Average Precision Mean Average Precision (mAP) summarizes the class-wise AP values by averaging them across all product categories:

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^C AP_c. \quad (5)$$

where C is the total number of product classes. The study reports both mAP@0.5 and mAP@0.5:0.95. The first uses an IoU threshold of 0.5, while the second averages performance over IoU thresholds ranging from 0.5 to 0.95.

Average Inference Latency Average inference latency records the time taken to process an image:

$$L_{avg} = \frac{1}{N} \sum_{i=1}^N (t_i^{end} - t_i^{start}). \quad (6)$$

where N is the number of evaluated images, t_i^{start} is the starting time, and t_i^{end} is the completion time for image i .

3.7 Error Analysis

Model errors were examined through confusion matrices, precision–recall curves, and qualitative detection examples. The confusion matrices were used to see whether one product was mistaken for another, whether the background generated false detections, and whether products were missed. Particular attention was given to visually similar packages, reflections, light-coloured backgrounds, and products placed close together.

Background false positives were treated as an operational error rather than only a detection statistic because an incorrect positive prediction can place an absent item on the bill. Background-only images were therefore included during training, and the confidence threshold was examined during evaluation. RGB and grayscale outputs were also compared to see whether removing colour reduced some forms of confusion or instead removed useful class information.

3.8 POS Integration

The detector with the most suitable combination of accuracy, latency, size, and false-positive behaviour was connected to a Python-based POS application. The application accepts an image or camera frame, applies the chosen preprocessing option, and sends the result to the trained detector. The predicted product class or GTIN is then used to retrieve the corresponding product record from the database.

The POS interface shows the detected product name, product identifier, quantity, and billing information. Increment, decrement, and delete controls are available so an operator can correct a wrong prediction before the transaction is completed. The prototype is therefore intended as a human-in-the-loop checkout assistant, not as a fully autonomous cashierless system. **Fig. 3** compares this workflow with a conventional barcode-based checkout process.

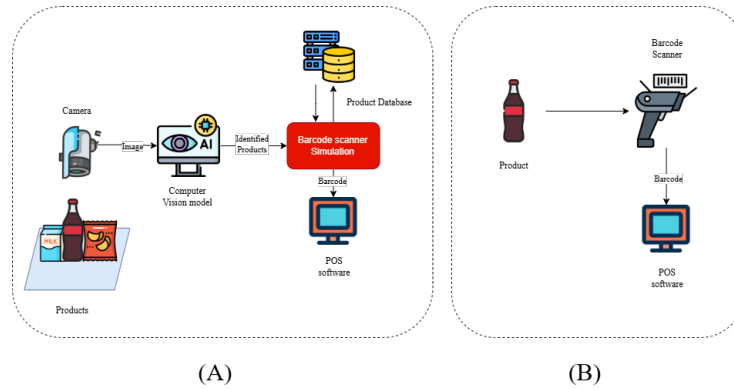


Fig. 3. Comparison of the proposed AI-powered grocery checkout workflow and traditional barcode-based checkout

4 Results and Discussion

4.1 Overall Model Performance

All four detectors achieved strong grocery-product recognition, as presented in **Table 4**. YOLOv8m, YOLOv11m, and RT-DETR obtained an mAP@0.5 of 0.995, while YOLOv12m achieved 0.994. Since the differences at IoU 0.5 were negligible, the stricter mAP@0.5:0.95 metric was more useful for distinguishing the models. YOLOv8m achieved the highest score of 0.935, followed by YOLOv11m at 0.924, RT-DETR at 0.919, and YOLOv12m at 0.913. Therefore, YOLOv8m provided the strongest overall localisation performance.

Table 4. OVERALL PERFORMANCE OF THE EVALUATED MODELS

Model	mAP@0.5	mAP@0.5:0.95	Size (MB)	Latency (ms/image)
YOLOv8m	0.995	0.935	49.6	696.3 $\pm$ 278.5
YOLOv11m	0.995	0.924	38.6	544.8 $\pm$ 40.2
YOLOv12m	0.994	0.913	38.8	558.6 $\pm$ 11.5
RT-DETR	0.995	0.919	63.1	1056.4 $\pm$ 169.7

Although YOLOv8m achieved the best localisation score, YOLOv11m offered the most efficient deployment profile, with the lowest latency and smallest model size. RT-DETR was the largest and slowest model despite its competitive mAP@0.5.

4.2 Confusion-Matrix Analysis

Fig. 4 presents the confusion matrices for (A) YOLOv8m, (B) YOLOv11m, (C) YOLOv12m, and (D) RT-DETR. All four matrices were strongly diagonal, indicating limited product-to-product confusion. YOLOv8m produced only two background false positives and one confusion between Clemon and Mojo. YOLOv11m and YOLOv12m each generated six background false positives, with minor confusion between the two digestive-biscuit classes.

RT-DETR produced 64 background false positives, including 33 incorrect detections of Fresh Toilet Tissue White. This weakness is particularly important in checkout applications because a false-positive prediction may add an absent product to the bill. Thus, the confusion-matrix analysis further supports YOLOv8m as the most dependable model for billing-sensitive deployment.

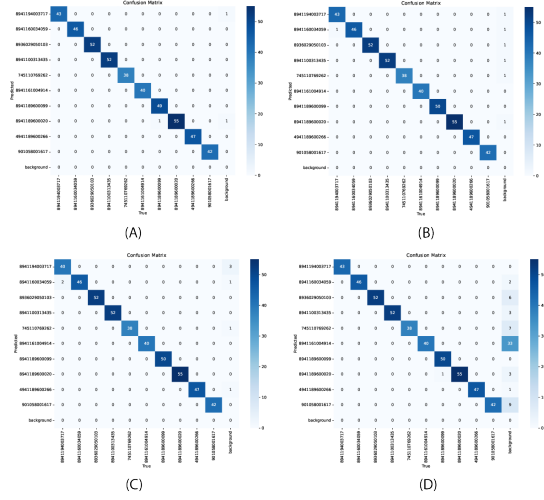


Fig. 4. Confusion Matrices of (A) YOLOv8m, (B) YOLOv11m, (C) YOLOv12m, and (D) RT-DETR

4.3 Class-Wise Performance

Table 5 shows that YOLOv8m achieved the highest mAP@0.5:0.95 for nine of the ten reported product classes. Clemon was the strongest class, reaching 0.978, followed by Good Knight at 0.957 and Mango Juice at 0.955. Chocolate Digestive Biscuit was the most difficult category across the three reported models. YOLOv11m performed best only for Nescafe, obtaining 0.921.

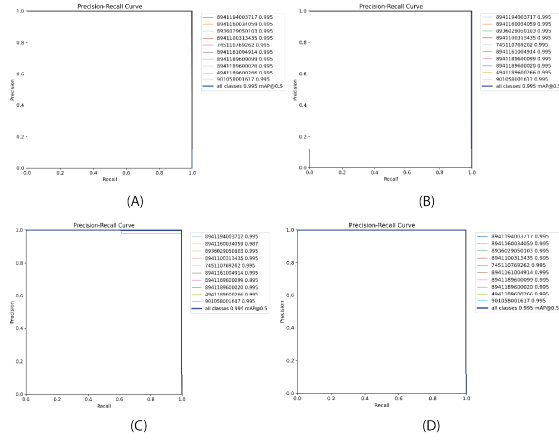
4.4 Precision–Recall Analysis

The precision–recall curves in **Fig. 5** remained close to the upper-right boundary for nearly all classes. Panels A, B, and D achieved an overall AP of 0.995 at IoU 0.5,

Table 5. CLASS-WISE mAP@0.5:0.95 PERFORMANCE

Product Class	YOLOv8m	YOLOv11m	RT-DETR
Chocolate Digestive Biscuit	0.899	0.891	0.887
BelleAme Digestive Biscuit	0.925	0.876	0.896
Mango Juice with Basil Seed	0.955	0.944	0.941
7up Drinks Bottle	0.928	0.906	0.906
Good Knight Refill	0.957	0.951	0.948
Fresh Toilet Tissue White	0.940	0.938	0.917
Clemon Can	0.978	0.968	0.971
Mojo Can	0.927	0.922	0.909
Speed Bottle	0.927	0.921	0.904
Nescafe Classic Glass Jar	0.913	0.921	0.909
Macro-average	0.935	0.924	0.919

while Panel C achieved 0.994. These nearly saturated curves confirm that all models maintained high precision across a broad recall range. However, the stricter results in **Table 4** and the background errors in **Fig. 4** provide stronger evidence for model selection.

**Fig. 5.** Precision–Recall Curves of (A) YOLOv8m, (B) YOLOv11m, (C) YOLOv12m, and (D) RT-DETR

Overall, YOLOv8m was the most reliable accuracy-oriented model, whereas YOLOv11m provided the best balance between compactness, speed, and detection performance.

4.5 Comparison with Other Studies

As shown in **Table 6**, earlier studies applied YOLO-based models to medical-device monitoring, drone imagery, retail shelf management, small-object detection, and industrial defect inspection.

Table 6. COMPARISON WITH OTHER RELATED WORKS

Existing Literature	Application	Classification/ Detection Method	Main Metric	Reported Performance
Mahmood et al. [2]	Medical-device parameter detection	pa- YOLOv8	Accuracy	99.7%
Maaroor and Bouhlel [8]	Drone-based object detection	object YOLOv8	Accuracy	0.91%*
Thirumurthy et al. [3]	Intelligent retail shelf management	YOLOv8n with transfer learning	mAP	69.0%
Madan and Reich [10]	Re- Small-object detection	detec- Optimized RT-DETR	mAP for small objects	51.3%
Madan Reich [10]	Small-object detection	detec- Baseline RT-DETR	mAP for small objects	38.9%
Gao et al. [13]	Weld-defect detection in X-ray images	YOLOv11s + P2	mAP@0.5	94.8%
Proposed System—YOLOv8m	Multi-product grocery detection and checkout	YOLOv8m	mAP@0.5 / mAP@0.5:0.95	99.5% / 93.5%

Mahmood et al. [2] reported an accuracy of 99.7% for real-time medical-device parameter detection, while Maaroor and Bouhlel [8] reported an accuracy value of 0.91, although the percentage interpretation requires verification. Thirumurthy et al. [3] obtained a mAP of 69% using YOLOv8n for retail shelf management. Madan and Reich [10] improved small-object detection performance from a baseline mAP of 38.9% to 51.3% using an optimized RT-DETR model. Gao et al. [13] achieved 93.07% precision, 94.8% mAP@0.5, and 72.01% mAP@0.5:0.95 using a YOLOv11s + P2 model for weld-defect detection.

In comparison, the proposed grocery-checkout system achieved mAP@0.5 values between 99.4% and 99.5% and mAP@0.5:0.95 values between 91.3% and 93.5% across YOLOv8m, YOLOv11m, YOLOv12m, and RT-DETR. YOLOv8m obtained the highest strict-IoU performance, with 99.5% mAP@0.5 and 93.5% mAP@0.5:0.95, whereas YOLOv11m provided the lowest latency and smallest model size. Unlike the compared studies, the proposed work also integrates multi-product recognition, GTIN-based database retrieval, and a human-in-the-loop POS interface for grocery checkout.

5 Conclusion

This work introduced an AI-based grocery checkout framework that combines multi-product detection, GTIN-based identification, database retrieval, and a human-in-the-loop POS interface. The system was evaluated on ten grocery products using YOLOv8m, YOLOv11m, YOLOv12m, and RT-DETR. All models achieved strong mAP@0.5 results ranging from 99.4% to 99.5%. YOLOv8m achieved the highest mAP@0.5:0.95 of 93.5%, while YOLOv11m provided the lowest latency of 544.8 ms per image and the smallest model size of 38.6 MB.

The confusion matrices showed limited product-to-product errors, although RT-DETR produced substantially more background-related false positives than the YOLO models. The RGB and grayscale comparison also indicated that grayscale preprocessing can correct some color-based errors but may introduce new misclassifications when color is important for recognition. Therefore, grayscale should remain an optional preprocessing method.

Overall, the proposed framework provides a strong foundation for snapshot-based grocery checkout assistance. Future work should use larger independent datasets, unseen

products and retail environments, improved background samples, calibrated confidence thresholds, and controlled usability and checkout-time evaluations.

References

1. Nouman Noor, M., Masab, M., Haneef, F., Hussain, M., Yaqoob, M., Mazhar, T., ... & Aldehim, G. (2026). An Effective Approach for Recognition of Crop Diseases Using Advanced Image Processing and YOLO v8. *Food Science & Nutrition*, 14(2), e71504.
2. Mahmood, M. S., Shoyaeb, M., Chowdhury, A., & Chowdhury, M. H. (2026). Automated health monitoring system using YOLOv8 for real-time device parameter detection. *Physical and Engineering Sciences in Medicine*, 49(1), 397-406.
3. Thirumurthy, B., Ravi, N., Kishore, Y. N., Parameswaran, L., & Vaiapury, K. (2026, March). Intelligent Retail Shelf Management Using Transfer Learning and YOLOv8n. In *2026 World Conference on Computational Science and Technology (WcCST)* (pp. 187-193). IEEE.
4. Hassani, I. B., Benhida, S., Lamii, N., Oqaidi, K., Ouiddad, A., & Ghiadi, S. (2026). From YOLO V1 to YOLO V11: comparative analysis of YOLO algorithm. *International Journal of Electrical and Computer Engineering (IJECE)*, 16(1), 450-462.
5. Muhammad, B. B., & Ahmad, M. R. (2026). Enhancing shoplifting detection precision through quantum-based bayesian optimization techniques. *Multimedia Tools and Applications*, 85(2), 105.
6. Baranwal, P., Mehra, T., Tiwari, N., & Shukla, S. (2026). AI-based autonomous shoplifting detection system using YOLOv8 and XGBoost. In *Artificial Intelligence and Sustainable Innovation* (pp. 544-549). CRC Press.
7. Sathya, J., Asha, V., Kalaivani, A., & Haslar, S. L. (2026, April). Grocery Detection Using YOLO v11 and RF-DETR (Nano) Object Detection Model. In *2026 6th International Conference on Trends in Material Science and Inventive Materials (ICTMIM)* (pp. 280-286). IEEE.
8. Maaroo, M. K. A., & Bouhlef, M. S. (2025). Real-Time Object Detection Using YOLO-8 Model: A Drone-Based Approach. *J Wirel Mob Netw Ubiquitous Comput Dependable Appl*, 16(1), 190-204.
9. Seth, Y., & Sivagami, M. (2025). Enhanced yolov8 object detection model for construction worker safety using image transformations. *IEEE Access*, 13, 10582-10594.
10. Madan, M., & Reich, C. (2025). strengthening small object detection in adapted RT-DETR through robust enhancements. *Electronics*, 14(19), 3830.
11. MUZAMMUL, M., Xuwei, L. I., & Li, X. (2025). Enhancing tiny object detection without fine tuning: Dynamic adaptive guided object inference slicing framework with latest YOLO models and RT-DETR transformer.
12. Nemati, N. (2025). Enhancing Maritime Object Detection in Real-Time with RT-DETR and Data Augmentation. *arXiv preprint arXiv:2510.07346*.
13. Gao, L., Liu, H., Gao, W., & He, J. (2026). YOLO11-Based Weld Defect Detection Method for X-Ray Images Integrating SIOU Bounding Box Regression and P2 Shallow Feature Enhancement. *Sensors*, 26(13), 4144.
14. Dahal, S. (2026). *Computer Vision and Deep Learning Approaches for Behavior Analysis and Welfare Assessment in Cage-Free Laying Hens* (Master's thesis, University of Georgia).
15. Ren, W., Zhang, S., & Zhou, Z. (2026). YOLO11-MGNB: lightweight real-time small object detection algorithm for UAV remote sensing images. *Journal of Real-Time Image Processing*, 23(1), 34.